

Блок	Вопросы, которые стоит рассмотреть	Место для заполнения
Общие характеристики курса	Название курса	Быстрый старт в FastAPI Python
	Длительность курса	5-6 дней
	Контактная работа (как реализуется работа студента и преподавателя совместно?)	Коммуникация с создателем курса в комментах

	Самостоятельная работа (что студент выполняет сам?)	Теорию, тесты, практические задания
Место дисциплины в учебной	Контроль (в каком формате проходит проверка знаний обучающихся по завершении курса?)	Тестовые вопросы, итоговый проект
	Дисциплины, которые студенты могли осваивать ранее	Программирование на Python, веб-разработка

учебной програм ме	Дисциплины, которые студенты могут проходить параллельно	Базы данных и СУБД, программное обеспечение, информационная безопасность
	Дисциплины, которые слушатели потенциально могут проходить в дальнейшем	Контейнеризация, продвинутое тестирование и мониторинг

	Зачем данный курс включен в образовательную программу? Какие образовательные цели программы закрывает данный курс?	Для создания современных веб-API, освоения инструментов Python-разработки, понимания принципов RESTful-архитектуры
	Компетенции	Проектировать и реализовывать веб-API, писать тесты для веб-приложений, управлять миграциями, развёртывать приложения

Цели и задачи дисциплины	Какие конкретно навыки/знания должны появиться в руках студента в после завершения данного курса?	Настраивать проект на FastAPI, обрабатывать параметры пути, запроса и тела запроса, использовать Pydantic для валидации данных, обрабатывать ошибки и исключения, подключать SQLite/PostgreSQL, писать тесты с использованием, развёртывать приложение с помощью Docker и ASGI-серверов
	Какие темы, касающиеся данного курса, студенты уже освоили в рамках других дисциплин?	Python, принципы ООП, базовые понятия сетей (клиент-сервер, запрос-ответ), основы реляционных баз данных и SQL

	Какие темы необходимо поднять в рамках курса, чтобы студенты могли перейти к изучению других курсов, связанных с данным?	Принципы работы асинхронного кода в Python, стандарты документирования API (OpenAPI/Swagger), подходы к логированию и мониторингу, базовые практики безопасности
--	--	--

	КТО целевая аудитория данного курса? (Аудитория должна быть сгруппирована по одному или нескольким признакам и описана по социально-демографически характеристикам)	Студенты технических специальностей, начинающие Python-разработчики, возраст 18-25 лет
--	---	---

	Каков уровень знаний целевой аудитории? (Необходимо сформулировать уровень знаний по теме (начальный, средний, продвинутый) и расшифровать, что на этом уровне знают/умеют студенты по	Начальный и начально-средний уровень
--	---	---

	Каковы боли целевой аудитории? У слушателей есть проблемы? (Нужно сформулировать боли основных сегментов в целевой аудитории, которые мы можем закрыть за счет прохождения	Нехватка структурированных материалов на русском языке, сложность самостоятельного перехода от теории к реальному коду без обратной связи, неопределённость в том, с чего начать изучение веб-разработки на Python
--	--	---

Целевая аудитория	Что пользователи получают от использования продукта?	Понимание FastAPI, опыт создания API с нуля, применение валидации, аутентификации, тестирования, сертификат, базу для дальнейшего профессионального роста и портфолио
	Почему данный продукт будет полезен слушателям?	Курс краткий, актуальный, гибкий, бесплатный и практичный
	Есть ли среди студентов те, чью точку зрения необходимо знать или кто требует особого подхода?	Студенты с опытом в других фреймворках, без опыта веб-разработки, студенты с ограниченным временем

	Каковы результаты учащихся?	Создать и развернуть веб-API: с маршрутами, валидацией, аутентификацией, подключением к БД и тестами
	Какие сложности могут возникнуть при работе с данной группой?	Разный уровень подготовки, технические сложности
	Что может привлечь внимание слушателей?	Возможность быстро получить востребованный навык, бесплатный формат и сертификат, ответы автора в комментариях, текстовый формат, каждый модуль заканчивается задачей, приближенной к реальной разработке

	Если слушателям не будет интересен этот курс, какой курс они будут посещать с удовольствием и использовать полученные знания из него?	Студенты могут предпочесть курсы по фронтенд-разработке или мобильной разработке, по анализу данных или машинному обучению
--	--	---

	Какие у слушателей могут быть вопросы и сомнения относительно цели курса или его содержания?	«Достаточно ли этого курса для трудоустройства?», «Почему текстовый формат, а не видео?», «Будет ли обратная связь по коду?», «Смогу ли я применить это в учебном проекте?»
	Кто реализует похожие курсы? (международный и отечественный опыт)	Udemy ("FastAPI - The Complete Course"), Coursera (специализации по Python Web Development), Real Python (статьи и tutorиалы по FastAPI), TestDriven.io (продвинутые гайды), Stepik (аналогичные курсы по Flask/Django), Яндекс.Практикум, Skillbox, OTUS (курсы по Python и микросервисам)

Исследование конкурентов	Какие инструменты/материалы/практики/темы я хочу позаимствовать у конкурентов? В чем будет пересекаться курс с конкурирующими?	Готовые шаблоны проектов и чек-листы для самопроверки, разборы типовых ошибок студентов в формате видео или статей. Темы, которые обязательно должны быть: работа с Pydantic, JWT-аутентификация, интеграция с БД, тестирование, деплой - это стандартный набор для любого курса по веб-API
	Чем курс будет отличаться от программ конкурентов?	Курс не перегружен теорией, фокус на практике, текстовый формат как преимущество: возможность учиться в тишине, перечитывать, делать пометки, бесплатный доступ с выдачей сертификата, ориентация на русскоязычную аудиторию